LeetCode 476: Number Complement

1. Problem Title & Link

Title: LeetCode 476: Number Complement

Link: https://leetcode.com/problems/number-complement/

2. Problem Statement (Short Summary)

Given a positive integer num, find its complement.

The complement of a number is obtained by flipping all bits in its binary representation:

0 → 1

1 → 0

Return the resulting decimal number.

Example

num = 5

Binary:

101

Flip every bit:

010

Decimal value:

2

Answer:

2

3. Examples (Input → Output)

Example 1

Input:

num = 5

Output:

2

Explanation:

5 = 101

Flip

010 = 2

Example 2

Input:

num = 1

Output:

0

Explanation:

1 = 1

Flip

0 = 0

Example 3

Input:

num = 10

Output:

5

Explanation:

10 = 1010

Flip

0101 = 5

4. Constraints

1 <= num < 2^31

Important:

Only flip the bits present in the number.

Do NOT consider leading zeros.

Wrong Thinking

For:

5

Binary:

00000101

Flipping all 8 bits:

11111010

This is NOT the expected answer.

Only flip:

101

to

010

5. Core Concept (Pattern / Topic)

Bit Manipulation

Secondary Topics:

- XOR
- Binary Representation
- Bit Masks

6. Thought Process (Step-by-Step Explanation)

Brute Force Thinking

Convert number to binary.

Example:

5 = 101

Flip every bit:

1 → 0

0 → 1

1 → 0

Result:

010

Convert back to decimal:

2

Works but requires binary conversion.

Optimized Thinking

Observe:

5 = 101

If we create:

111

and XOR it with:

101

then:

101

111

010

which is exactly the complement.

Key Idea

Create a mask consisting of all 1s having the same length as the binary representation.

Example:

num = 5

Binary = 101

Mask = 111

Then:

answer = num ^ mask

7. Visual / Intuition Diagram (ASCII Diagram)

```
Example:
num = 5
Binary:
101
Mask:
111
XOR:
101
111
---
010
Answer:
2

Another Example:
10
Binary:
1010
Mask:
1111
XOR:
1010
1111
----
0101
```

```
Answer:
5
```

8. Pseudocode (Language Independent)

```
mask = 1

while mask < num

    mask = (mask << 1) | 1

return num XOR mask
```

9. Code Implementation

Python

```
class Solution:
    def findComplement(self, num: int) -> int:

        mask = 1

        while mask < num:
            mask = (mask << 1) | 1

        return num ^ mask
```

Java

```
class Solution {

    public int findComplement(int num) {

        int mask = 1;

        while(mask < num) {
            mask = (mask << 1) | 1;
        }

        return num ^ mask;
    }
}
```

10. Time & Space Complexity

Time Complexity

O(log n)

Reason:

Mask grows one bit at a time.

Space Complexity

O(1)

Only a few variables are used.

11. Common Mistakes / Edge Cases

Mistake 1

Using:

~num

directly.

Example:

~5

Result:

-6

Not expected.

Why?

Because computers store integers using fixed-width binary representations.

Mistake 2

Flipping leading zeros.

For:

5

Only:

101

should be flipped.

Not:

00000101

Edge Cases

Smallest Value

Input:

1

Binary:

1

Complement:

0

Output:

0

Power of Two

Input:

8

Binary:

1000

Complement:

0111

Output:

7

12. Detailed Dry Run (Step-by-Step Table)

Input:

num = 5

Binary:

101

Build Mask

Initial:

mask = 1

Step	mask
1	1
2	3 (11)
3	7 (111)

Now:

mask >= num

Stop.

XOR

num = 101

mask = 111

101

111

010

Result:

2

13. Common Use Cases (Real-Life / Interview)

Bitwise Data Processing

Network packets

Flags

Permissions

Image Processing

Invert pixels

Example:

Black → White

White → Black

Embedded Systems

Sensor state inversion

Hardware control bits

Competitive Programming

Frequently appears in:

Bitmask problems

XOR operations

Binary transformations

14. Common Traps (Important!)

Trap 1

Using:

~num

instead of creating a mask.

Trap 2

Forgetting that:

Leading zeros are ignored.

Trap 3

Building wrong mask.

Example:

num = 10

1010

Need:

1111

not:

111

Trap 4

Not understanding XOR.

Remember:

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

XOR with 1 flips a bit.

15. Builds To (Related LeetCode Problems)

Progression:

LC 476 - Number Complement

↓

LC 1009 - Complement of Base 10 Integer

↓

LC 231 - Power of Two

↓

LC 191 - Number of 1 Bits

↓

LC 338 - Counting Bits

↓

LC 190 - Reverse Bits

These build strong Bit Manipulation fundamentals.

16. Alternate Approaches + Comparison

Approach	Time	Space	Interview Rating
Binary String Conversion	O(log n)	O(log n)	Easy
Bit-by-Bit Flip	O(log n)	O(1)	Good
Mask + XOR	O(log n)	O(1)	Best

Approach 1: String Conversion

binary = bin(num)[2:]

Flip characters.

Simple but uses extra space.

Approach 2: Manual Bit Processing

Check each bit individually.

More code.

Approach 3: Mask + XOR

num ^ mask

Elegant and commonly expected in interviews.

17. Why This Solution Works (Short Intuition)

A mask of all 1s has the same bit length as the number.

When XOR is applied:

0 XOR 1 = 1

1 XOR 1 = 0

Every bit gets flipped, producing exactly the complement.

18. Variations / Follow-Up Questions

Follow-Up 1

Find complement of every number in an array.

Example:

[5,1,10]

Output:

[2,0,5]

Follow-Up 2

Invert only odd-position bits.

Example:

1010

Flip positions:

1,3,5...

only.

Follow-Up 3

Count how many bits changed after complementing.

Example:

5 = 101

Complement:

010

Changed bits:

3

Follow-Up 4

Find complement using only arithmetic operations without bitwise operators.

Interview Takeaway

This problem teaches one of the most important Bit Manipulation patterns:

Create a mask

+

Use XOR to flip bits

The key formula to remember is:

Complement = Number XOR Mask

where the mask consists of all 1s covering the binary length of the number.